Báo cáo hướng làm và phân tích chi tiết hệ thống website thương mại điện tử thời trang

Nov 4, 2026 · @Ng Tr Quang Minh

Tổng quan đề tài và định hướng giải pháp

Đề tài xây dựng một hệ thống thương mại điện tử thời trang theo kiến trúc **ứng dụng web render phía máy chủ (SSR) theo mô hình MVC**, trong đó giao diện được dựng bằng view engine EJS ngay trên máy chủ, còn Backend dùng **Node.js** cung cấp **RESTful API** và cơ sở dữ liệu **NoSQL Document (MongoDB)**. Trọng tâm kỹ thuật của đồ án gồm bốn bài toán: phân quyền **RBAC**, bộ lọc đa thuộc tính trên **biến thể sản phẩm**, **kiểm soát đồng thời tồn kho** (Concurrency Control) và **bảo mật theo chiều sâu** (Defense in Depth).

Mục tiêu và phạm vi

- **Mục tiêu nghiệp vụ:** cho phép khách duyệt, lọc, chọn đúng biến thể (Size – Màu), đặt hàng và theo dõi đơn; cho phép quản trị viên vận hành danh mục, sản phẩm, đơn hàng và xem báo cáo.
- **Mục tiêu kỹ thuật:** đảm bảo không bán vượt tồn kho, không lộ dữ liệu nhạy cảm, truy vấn danh mục nhanh dưới tải thực tế.
- **Ngoài phạm vi (đề xuất ghi rõ trong thuyết minh):** hệ thống gợi ý bằng học máy, đa kho vật lý, đa ngôn ngữ/đa tiền tệ, tích hợp trực tiếp hãng vận chuyển (chỉ mô phỏng trạng thái).

Lựa chọn công nghệ và lý do

Thành phần	Lựa chọn	Lý do chọn (so với phương án thay thế)
Runtime Backend	Node.js + Express	Mô hình I/O không chặn phù hợp tác vụ nhiều truy vấn ngắn; hệ sinh thái middleware phong phú; EJS cho phép viết mẫu giao diện bằng cú pháp JavaScript quen thuộc, giúp giảm chi phí học.

Thành phần	Lựa chọn	Lý do chọn (so với phương án thay thế)
Cơ sở dữ liệu	MongoDB (NoSQL Document)	Sản phẩm thời trang có thuộc tính biến thiên theo loại (áo, giày, phụ kiện); lược đồ linh hoạt và mảng lồng nhau phù hợp hơn bảng quan hệ cố định. Đổi lại, ràng buộc toàn vẹn phải do tầng ứng dụng đảm nhiệm.
Xác thực	JWT trong HttpOnly Cookie	Phi trạng thái (stateless) nên dễ mở rộng ngang; HttpOnly ngăn mã JavaScript đọc token, giảm rủi ro đánh cắp qua XSS so với lưu token ở localStorage.
Giao diện (View)	Server-Side Rendering với view engine EJS (trên Express)	HTML hoàn chỉnh được dựng sẵn ở máy chủ nên trang hiện nhanh và thân thiện SEO (quan trọng với cửa hàng); không cần bộ công cụ build riêng; logic và kiểm tra quyền nằm trọn ở máy chủ. JavaScript phía trình duyệt chỉ dùng cho các tương tác nhỏ.
Hạ tầng	MongoDB Atlas, Render/Railway	Dịch vụ quản trị sẵn, có gói miễn phí/giá thấp phù hợp đồ án; tách hai môi trường triển khai độc lập (cơ sở dữ liệu và ứng dụng web).

> **[Cần thẩm định]** Khung sườn gốc dùng SPA; theo yêu cầu mới, báo cáo đã chuyển giao diện sang SSR với EJS. Chương 4 và các mục liên quan đến CORS, cookie và triển khai đã được cập nhật tương ứng.

Chương 1. Phân tích yêu cầu hệ thống

Chương này xác định **ai** được làm **gì** trên hệ thống (tác nhân và RBAC) và hệ thống phải **cung cấp chức năng nào** (use case theo phân hệ). Đây là cơ sở để suy ra thiết kế ở Chương 2.

1.1. Tác nhân và ma trận phân quyền RBAC

Hệ thống áp dụng mô hình **RBAC (Role-Based Access Control)** với ba vai trò. Phân quyền theo vai trò được chọn thay vì phân quyền theo từng người dùng vì số lượng chức năng nhỏ, ranh giới vai trò rõ ràng, dễ kiểm thử và dễ giải thích trong thuyết minh.

- **Khách vãng lai (Guest):** tác nhân chưa định danh. Được truy cập tài nguyên công khai, tra cứu danh mục, tìm kiếm theo từ khóa, áp bộ lọc (Size, Màu, khoảng giá) và dùng giỏ hàng tạm lưu trong phiên (session) phía máy chủ.
- **Khách hàng (Customer):** đã đăng nhập. Kế thừa quyền của Guest; có giỏ hàng lưu bền trên cơ sở dữ liệu; đặt hàng, thanh toán; tra cứu lịch sử và trạng thái đơn; yêu cầu hủy đơn trong điều kiện cho phép.
- **Quản trị viên (Admin):** đặc quyền cao nhất. Quản lý danh mục, sản phẩm và biến thể; điều phối trạng thái đơn hàng; quản lý tài khoản và phân quyền; xem báo cáo kinh doanh và cảnh báo tồn kho.

Ma trận phân quyền (✓ = được phép; ✗ = bị từ chối; * = chỉ trên dữ liệu của chính mình):

Chức năng	Guest	Customer	Admin
Xem danh mục, sản phẩm, tìm kiếm, lọc	✓	✓	✓
Giỏ hàng tạm (phiên máy chủ)	✓	✓	✓
Giỏ hàng đồng bộ trên máy chủ	✗	✓*	✗
Đặt hàng, thanh toán	✗	✓*	✗
Xem và hủy đơn của mình	✗	✓*	✓
Cập nhật hồ sơ cá nhân	✗	✓*	✓*
CRUD danh mục, sản phẩm, biến thể	✗	✗	✓
Chuyển trạng thái đơn hàng	✗	✗	✓
Quản lý tài khoản, phân quyền	✗	✗	✓
Xem báo cáo, cảnh báo tồn kho	✗	✗	✓

Lập luận bổ sung. Dấu * đòi hỏi kiểm tra **quyền sở hữu** (ownership check) ở tầng Controller, không chỉ kiểm tra vai trò: một Customer không được đọc hay hủy đơn của Customer khác dù cùng vai trò. Đây là lỗi thường gặp (IDOR – Insecure Direct Object Reference) và nên được nêu thành một yêu cầu phi chức năng riêng.

1.2. Đặc tả use case theo phân hệ

Hệ thống chia thành bốn phân hệ. Mỗi use case dưới đây nêu mục đích và luồng chính.

Phân hệ Xác thực và Bảo mật (Authentication Module)

- **Đăng ký:** nhận dữ liệu → kiểm tra Email chưa tồn tại → băm mật khẩu một chiều → lưu tài khoản với vai trò mặc định Customer.
- **Đăng nhập:** đối chiếu thông tin xác thực → cấp token phiên → trả về dưới dạng cookie bảo mật. Hỗ trợ đăng nhập liên kết **OAuth2 với Google** như phương án thay thế, giúp giảm ma sát đăng ký.
- **Quản trị tài khoản:** quên mật khẩu (cấp lại quyền truy cập qua liên kết có thời hạn gửi email), cập nhật hồ sơ, đổi mật khẩu.

Phân hệ Quản lý Sản phẩm (Product Catalog Module)

- **Truy vấn bộ sưu tập:** đọc danh sách có phân trang và sắp xếp động (mới nhất, giá tăng/giảm).
- **Lọc ma trận nâng cao:** kết hợp nhiều điều kiện cùng lúc gồm danh mục, Size, Màu, chất liệu, khoảng giá và từ khóa.
- **Tra cứu biến thể:** trả về tồn kho của từng cặp Size – Màu để giao diện biết nút nào còn hàng.

Phân hệ Giao dịch và Đơn hàng (Order Processing Module)

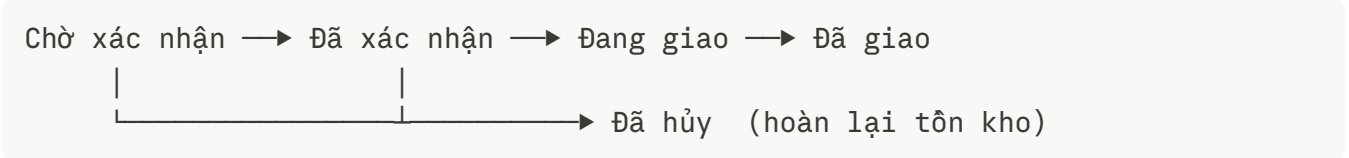
- **Quản lý giỏ hàng:** tính tổng giá trị, kiểm tra số lượng so với tồn kho của đúng biến thể.
- **Khởi tạo đơn hàng:** thu thập thông tin giao nhận; thanh toán trực tuyến hoặc COD (thanh toán khi nhận hàng).
- **Kiểm soát vòng đời đơn:** cho phép hủy khi đơn chưa bàn giao cho đơn vị vận chuyển; khi hủy phải hoàn lại tồn kho.

Phân hệ Quản trị Hệ thống (Admin Control Module)

- **CRUD nghiệp vụ:** danh mục, sản phẩm, khách hàng.
- **Điều phối đơn hàng:** chuyển trạng thái theo luồng tuyến tính cho phép.
- **Trực quan hóa dữ liệu:** chỉ số kinh doanh, biểu đồ tăng trưởng, danh sách biến thể chạm ngưỡng cảnh báo tồn kho.

1.3. Luồng trạng thái đơn hàng (đề xuất bổ sung)

Khung sườn gốc chỉ nêu bốn nhóm trạng thái. Để phục vụ ràng buộc "chỉ hủy khi chưa bàn giao vận chuyển", báo cáo đề xuất máy trạng thái hữu hạn sau:



- Chuyển trạng thái chỉ đi **tiến**; chuyển lùi bị chặn ở tầng nghiệp vụ.

- Hủy chỉ hợp lệ ở hai trạng thái đầu; từ "Đang giao" trở đi chỉ Admin xử lý qua quy trình hoàn trả riêng (ngoài phạm vi).

> **[Cần thẩm định]** Trạng thái "Đã xác nhận" là bổ sung của tôi so với khung sườn gốc (Chờ xác nhận / Đang giao / Đã giao / Đã hủy). Nếu giảng viên muốn giữ đúng bốn trạng thái, có thể gộp "Đã xác nhận" vào "Chờ xác nhận" nhưng sẽ mất mốc thời điểm trừ kho/chuẩn bị hàng.

Chương 2. Thiết kế kiến trúc hệ thống

2.1. Kiến trúc phân tầng kết hợp RESTful API

Backend theo **Layered Architecture**, mỗi tầng chỉ gọi tầng liền kề bên dưới:

```
Trình duyệt
   |  HTTP request (GET, POST, fetch)
   ▼
Routes        – tuyến giao diện (trả HTML) và tuyến /api (trả JSON)
   ▼
Middlewares   – Helmet, phiên, JWT, RBAC, CSRF, làm sạch dữ liệu, kiểm tra
đầu vào
   ▼
Controllers   – xử lý nghiệp vụ, điều phối luồng
   ├──► Models ──► MongoDB Atlas      (đọc/ghi dữ liệu)
   └──► Views (EJS) ──► HTML          (dựng trang trả về trình duyệt)
```

Tầng **View (EJS)** nằm ngay trong ứng dụng Node.js. Tuyến giao diện trả HTML hoàn chỉnh; tuyến dưới tiền tố /api trả JSON cho các tương tác nhỏ và cho kiểm thử. Cả hai dùng chung Controller và Model nên nghiệp vụ không bị nhân đôi.

Tại sao chọn phân tầng?

- **Tách mối quan tâm (Separation of Concerns):** sửa quy tắc bảo mật không động đến nghiệp vụ; đổi cơ sở dữ liệu không động đến Routes.
- **Kiểm thử độc lập:** Controller có thể kiểm thử với Model giả lập.
- **Phù hợp quy mô đồ án:** đơn giản hơn microservices nhưng vẫn cho phép tách thành dịch vụ sau này (ví dụ tách phân hệ đơn hàng).

Tại sao chọn RESTful? Tài nguyên (sản phẩm, đơn hàng, người dùng) ánh xạ tự nhiên sang URL danh từ; dùng đúng động từ HTTP (GET đọc, POST tạo, PUT/PATCH sửa, DELETE xóa) và mã trạng thái chuẩn (200, 201, 400, 401, 403, 404, 409), giúp phần JavaScript nhẹ ở trình duyệt và công cụ kiểm thử như Postman dễ dùng (áp dụng cho các tuyến /api).

2.2. Vòng đời Request (Request Lifecycle)

Mọi request đi qua chuỗi xử lý cố định. Thứ tự này là **có chủ đích**: các rào chắn rẻ và mang tính loại trừ được đặt trước, tác vụ tốn kém đặt sau.

Bước	Giai đoạn	Thành phần	Mục đích và lý do đặt ở vị trí này
1	Tiếp nhận	Cổng vào (Gateway/máy chủ Express)	Nhận HTTP Request, phân tích phần thân (dữ liệu biểu mẫu và JSON) với giới hạn kích thước để chống request quá lớn.
2	Rào cản mạng	**CORS** + **Helmet**	CORS chặn request từ nguồn gốc lạ gọi vào các tuyến /api (giao diện và API cùng một máy chủ nên phần lớn request là cùng nguồn gốc); Helmet đặt các tiêu đề bảo mật HTTP (chống clickjacking, ép MIME đúng loại…). Chặn sớm các request không hợp lệ về nguồn gốc.
3	Thẩm định danh tính	**JWT Authentication Middleware**	Đọc token từ HttpOnly Cookie, kiểm chữ ký và hạn dùng, gắn danh tính và vai trò vào request. Token sai/hết hạn → 401.
4	Phân quyền	Authorization Middleware (RBAC)	So vai trò với quyền của route. Không đủ quyền → 403. Tách khỏi bước 3 vì "là ai" khác với "được làm gì".
5	Làm sạch dữ liệu	**Sanitize Middleware**	Loại bỏ chuỗi script độc hại trong payload trước khi vào nghiệp vụ.
6	Thực thi nghiệp vụ	Controller → Model → View (EJS)	Tính toán, truy vấn cơ sở dữ liệu, dựng trang HTML bằng EJS (tuyến giao diện) hoặc trả JSON chuẩn hóa (tuyến /api).
7	Xử lý lỗi	Error-handling Middleware	Gom mọi lỗi về một định dạng phản hồi; không lộ chi tiết nội bộ (stack trace) ra trình duyệt; tuyến giao diện hiển thị trang lỗi EJS, tuyến /api trả JSON lỗi.

Lập luận bổ sung và các điểm cải tiến so với khung sườn gốc

- Khung sườn gốc không có bước **phân quyền (4)**, **kiểm tra đầu vào theo lược đồ** và **xử lý lỗi tập trung (7)**; tôi bổ sung vì ba thành phần này là phần cốt lõi của một API chuẩn sản xuất.
- **Giới hạn tần suất (rate limiting)** nên đặt ở bước 2, đặc biệt cho đăng nhập và quên mật khẩu, để giảm dò mật khẩu hàng loạt (brute force).
- Cookie xác thực nên đặt cờ **Secure** và **SameSite**; vì giao diện và API cùng một nguồn gốc nên dùng được SameSite ở mức Lax hoặc Strict, và cần cấu hình tin cậy proxy khi chạy sau hạ tầng của nền tảng. Mọi biểu mẫu POST cần kèm cơ chế chống **CSRF** (ví dụ token CSRF gắn vào từng biểu mẫu).

> **[Cần thẩm định]** Khung sườn gốc dùng DOMPurify cho Sanitize Middleware. DOMPurify vốn làm sạch HTML ở trình duyệt và cần môi trường DOM (ví dụ jsdom) khi chạy trên Node.js. Với dữ liệu thuần văn bản như tên, địa chỉ, đánh giá, các thư viện chuyên cho Node (như xss hoặc sanitize-html) thường nhẹ hơn. Quan trọng hơn, **thoát ký tự theo ngữ cảnh khi hiển thị** (output encoding) ở tầng View mới là tuyến phòng thủ chính chống XSS (cú pháp xuất mặc định của EJS tự thoát ký tự); làm sạch đầu vào chỉ là lớp bổ sung. Tôi giữ DOMPurify để bám khung sườn nhưng nêu rõ hạn chế này để tránh bị hỏi vặn khi bảo vệ.

2.3. Thiết kế mô hình dữ liệu (NoSQL Document)

Tại sao NoSQL Document? Một sản phẩm thời trang luôn được đọc **cùng với toàn bộ biến thể** của nó (trang chi tiết cần cả danh sách Size – Màu – tồn kho). Nhúng biến thể vào chính tài liệu sản phẩm (Embedded Data) cho phép lấy đủ dữ liệu bằng **một** truy vấn, không cần phép nối (JOIN) như mô hình quan hệ. Đổi lại, cập nhật tồn kho của một biến thể phải dùng toán tử cập nhật trên phần tử mảng (xem Chương 3), và mảng biến thể không nên tăng vô hạn.

Quy tắc chọn **nhúng** hay **tham chiếu** áp dụng trong báo cáo:

- **Nhúng** khi dữ liệu con luôn đọc cùng cha, số lượng nhỏ và có giới hạn (biến thể của sản phẩm; chi tiết dòng hàng của đơn).
- **Tham chiếu** khi dữ liệu con tồn tại độc lập hoặc tăng không giới hạn (đơn hàng của người dùng; đánh giá sản phẩm).

Thực thể	Các trường chính	Ghi chú thiết kế
User	Họ tên; Email (duy nhất); số điện thoại; địa chỉ mặc định; mật khẩu băm kèm muối; vai trò (role)	Email có chỉ mục duy nhất (unique index). Mật khẩu chỉ lưu giá trị băm bằng thuật toán có muối như bcrypt; không bao giờ trả trường này ra API.

Thực thể	Các trường chính	Ghi chú thiết kế
Category	Tên; slug (đường dẫn thân thiện SEO); danh mục cha; trạng thái hiển thị	Cây danh mục cha/con qua tham chiếu danh mục cha. Slug có chỉ mục duy nhất, dùng làm URL.
Product	Tiêu đề; mô tả; giá (kiểu số); mảng URL ảnh trên CDN; tham chiếu Category; chất liệu; **mảng variants**	Ảnh chỉ lưu URL, không lưu tệp nhị phân trong cơ sở dữ liệu.
Variant (nhúng trong Product)	Size; Color; **stock**	Mỗi phần tử là một cặp Size – Màu độc lập với tồn kho riêng. Nên có mã định danh biến thể (SKU) để tham chiếu từ đơn hàng.
Order	Tham chiếu User; thông tin giao nhận (Tỉnh/Thành, Quận/Huyện, Phường/Xã, địa chỉ chi tiết); tổng thanh toán; phương thức và trạng thái thanh toán; trạng thái đơn	Người dùng tham chiếu bằng khóa ngoại logic (MongoDB không ép ràng buộc này, tầng ứng dụng phải đảm bảo).
OrderItem (nhúng trong Order)	Tham chiếu Product; Size/Màu đã chọn; số lượng; **giá tại thời điểm mua**	Chụp lại giá (price snapshot) để sổ sách đúng dù giá gốc đổi sau này. Nên chụp thêm tên và ảnh đại diện để lịch sử đơn không vỡ khi sản phẩm bị sửa hoặc xóa.

Chỉ mục (index) đề xuất để đạt hiệu năng truy vấn:

- Chỉ mục duy nhất: Email người dùng, slug danh mục, slug sản phẩm.
- Chỉ mục hợp thành cho lọc danh mục: (category, price). Chỉ mục đa khóa (multikey) trên (variants.size, variants.color) cho bộ lọc biến thể.
- Chỉ mục (user, thời điểm tạo giảm dần) trên Order để tra lịch sử đơn của khách.
- Chỉ mục văn bản (text index) nếu muốn tìm kiếm theo từ khóa tốt hơn biểu thức chính quy (xem phần thẩm định).

Lập luận bổ sung. Giỏ hàng lưu bền của Customer có thể là một tài liệu riêng (Cart) tham chiếu User, hoặc một mảng nhúng trong User. Tôi khuyến nghị **tài liệu Cart riêng** để thay đổi giỏ hàng (rất thường xuyên) không làm ghi lại tài liệu User.

> **[Cần thẩm định]** Khung sườn gốc chưa nêu thực thể **Cart** và **Review** dù giao diện có form đánh giá sản phẩm (Chương 5). Tôi đã thêm hai thực thể này vào thiết kế đề xuất; nếu cần giữ đúng khung sườn, có thể ghi chú là "mở rộng".

Chương 3. Quy trình hiện thực hóa Backend (Node.js)

3.1. Khởi tạo và thiết lập môi trường nền tảng

- **Cấu trúc dự án:** tổ chức theo tầng đã nêu ở Chương 2 (routes, middlewares, controllers, models, utils, config); quản lý phụ thuộc bằng **npm** với tệp khóa phiên bản để môi trường dựng lại đồng nhất.
- **Cách ly cấu hình nhạy cảm vào tệp biến môi trường (.env):** chuỗi kết nối cơ sở dữ liệu, khóa bí mật ký JWT, khóa cổng thanh toán, thông tin dịch vụ email. *Lý do:* tách cấu hình khỏi mã nguồn để cùng một mã chạy ở nhiều môi trường (Dev, Production) và không rò rỉ bí mật lên kho mã. Tệp .env phải nằm trong danh sách bỏ qua của hệ quản lý phiên bản, kèm một tệp mẫu không chứa giá trị thật.
- **Cấu hình lõi của ứng dụng:** khai báo EJS làm view engine và thư mục tài nguyên tĩnh; Helmet cho tiêu đề HTTP; bộ phân tích dữ liệu biểu mẫu và JSON có giới hạn kích thước; bộ đọc cookie và kho phiên (session); cơ chế chống CSRF; CORS chỉ bật cho các tuyến /api khi có nguồn gốc khác gọi vào. Kiểm tra biến môi trường bắt buộc ngay lúc khởi động; thiếu thì dừng ứng dụng thay vì chạy với cấu hình lỗi.

3.2. Phân hệ Xác thực (Auth Logic)

Luồng đăng ký

1. Nhận dữ liệu và kiểm tra định dạng (Email hợp lệ, mật khẩu đủ độ mạnh).
2. Truy vấn tìm Email đã tồn tại; nếu có, trả mã **409 Conflict**.
3. Băm mật khẩu một chiều bằng thuật toán băm có muối và hệ số chi phí điều chỉnh được (bcrypt hoặc argon2).
4. Lưu tài khoản với vai trò mặc định là Customer. Vai trò **không bao giờ** được nhận từ dữ liệu client gửi lên.

Luồng đăng nhập

1. Tìm người dùng theo Email; so khớp mật khẩu nhập vào với giá trị băm đã lưu (phép so sánh băm, không giải mã).
2. Sai thông tin → trả thông báo chung "Email hoặc mật khẩu không đúng", không tiết lộ trường nào sai (chống dò tài khoản).
3. Đúng → tạo **JWT** chứa mã người dùng và vai trò, có thời hạn ngắn, ký bằng khóa bí mật.

4. Đặt token vào cookie với các cờ **HttpOnly**, **Secure**, **SameSite**.

Lập luận bổ sung. Token truy cập thời hạn ngắn (ví dụ 15–30 phút) kết hợp token làm mới (refresh token) thời hạn dài giúp hạn chế thiệt hại khi lộ token mà vẫn giữ trải nghiệm đăng nhập liền mạch. Đây là cải tiến so với khung sườn (chỉ nêu một token có hạn).

3.3. Phân hệ Bộ lọc ma trận nâng cao (Advanced Filter Logic)

Bộ lọc nhận các tham số trên URL và **lắp ráp động** một điều kiện truy vấn duy nhất: chỉ thêm điều kiện khi tham số có mặt.

Tham số URL	Điều kiện truy vấn tạo ra	Lưu ý
Mã danh mục	Sản phẩm thuộc danh mục (và danh mục con nếu cần)	Dùng slug hoặc mã định danh.
Khoảng giá X–Y	Giá lớn hơn hoặc bằng X và nhỏ hơn hoặc bằng Y	Kiểm tra X nhỏ hơn hoặc bằng Y; giá trị không phải số bị loại.
Từ khóa	Biểu thức chính quy tìm tương đối, không phân biệt hoa thường, trên tiêu đề	**Phải thoát ký tự đặc biệt** của từ khóa trước khi dựng biểu thức, nếu không có nguy cơ tiêm biểu thức và treo truy vấn (ReDoS).
Size và Màu	Tồn tại **cùng một** biến thể khớp Size, khớp Màu và còn hàng	Xem giải thích bên dưới.

Tìm sâu vào mảng lồng nhau với điều kiện kép. Đây là điểm dễ sai nhất. Nếu đặt điều kiện Size và điều kiện Màu như hai điều kiện rời rạc trên mảng biến thể, hệ thống có thể khớp một sản phẩm có biến thể (Size M, hết hàng) và một biến thể khác (Màu Đỏ, còn hàng) dù không tồn tại biến thể "M – Đỏ – còn hàng". Giải pháp là dùng toán tử **khớp phần tử mảng (element match)** để buộc ba điều kiện (Size, Màu, tồn kho lớn hơn 0) cùng áp lên **một phần tử**.

Phân trang. Chỉ số bỏ qua = (trang hiện tại − 1) × số bản ghi mỗi trang; giới hạn = số bản ghi mỗi trang. Phản hồi trả kèm tổng số bản ghi và tổng số trang để mẫu EJS dựng thanh phân trang. Giới hạn trên của số bản ghi mỗi trang phải được ép ở Backend để tránh yêu cầu tải toàn bộ dữ liệu.

> **[Cần thẩm định]** Phân trang theo bỏ qua/giới hạn chậm dần khi trang rất sâu vì cơ sở dữ liệu vẫn phải duyệt qua các bản ghi bị bỏ. Với danh mục cỡ đồ án thì chấp nhận được; nếu bổ sung cuộn vô hạn bằng JavaScript phía trình duyệt, **phân trang theo con trỏ (cursor-based)** tối ưu hơn. Ngoài ra biểu thức chính quy không tận dụng được chỉ mục khi khớp ở giữa chuỗi; chỉ mục văn bản (text index) hoặc Atlas Search là phương án tối ưu hơn.

3.4. Phân hệ Xử lý đơn hàng và cơ chế khóa kho (Concurrency Stock Control)

Bài toán. Hai khách cùng đặt biến thể "Áo thun – M – Đen" khi chỉ còn 1 sản phẩm. Nếu hệ thống **đọc tồn kho, kiểm tra, rồi mới ghi giảm** thành ba bước tách rời, cả hai có thể cùng đọc "còn 1" và cùng thành công, dẫn đến bán vượt kho (race condition, còn gọi lỗi "đọc – sửa – ghi").

Giải pháp: kiểm tra và trừ kho trong một phép toán nguyên tử (Atomic Operator). Thay vì đọc rồi ghi, hệ thống gửi **một** lệnh cập nhật có điều kiện: "trừ số lượng mua khỏi tồn kho của đúng biến thể **chỉ khi** tồn kho hiện tại lớn hơn hoặc bằng số lượng mua". MongoDB đảm bảo thao tác trên một tài liệu là nguyên tử, nên chỉ một trong hai request thắng; request còn lại nhận kết quả "không có tài liệu nào được sửa" và được hiểu là hết hàng. Cách này cũng nhanh hơn khóa bi quan (pessimistic lock) vì không giữ khóa nào giữa các bước.

Luồng đặt hàng tuần tự

1. Nhận danh sách dòng hàng từ giỏ hàng; **tính lại giá từ cơ sở dữ liệu**, không tin giá do client gửi.
2. Với từng dòng hàng, thực hiện phép trừ kho có điều kiện nguyên tử trên đúng biến thể.
3. **Đủ hàng** (mọi dòng đều trừ thành công): tạo bản ghi Đơn hàng với trạng thái *Chờ xác nhận*, chụp lại giá từng dòng, xóa các dòng đã mua khỏi giỏ hàng.
4. **Thiếu hàng** (có dòng trừ không thành công): dừng tiến trình, **hoàn lại** các dòng đã trừ trước đó, trả mã lỗi (409 Conflict) kèm thông báo biến thể nào vượt giới hạn kho.
5. **Hủy đơn** hợp lệ thì cộng trả tồn kho bằng phép cộng nguyên tử tương ứng.

Lập luận bổ sung: tính nhất quán của đơn nhiều dòng. Phép nguyên tử trên từng tài liệu chưa đảm bảo "tất cả hoặc không dòng nào" khi đơn có nhiều dòng thuộc nhiều sản phẩm. Hai hướng xử lý: (a) **giao dịch đa tài liệu (multi-document transaction)** của MongoDB, đòi hỏi bộ bản sao (replica set) mà Atlas cung cấp sẵn; (b) bù trừ thủ công như bước 4 ở trên. Với đồ án, (a) gọn và đúng đắn hơn; (b) là phương án dự phòng.

> **[Cần thẩm định]** (1) Khung sườn gốc mô tả "vòng lặp kiểm tra điều kiện tiên quyết rồi mới trừ"; nếu hiểu là kiểm tra riêng rồi trừ riêng thì vẫn còn nguy cơ race condition, nên tôi diễn giải lại thành một thao tác có điều kiện duy nhất. (2) Việc trừ kho ngay khi đặt (thay vì khi xác nhận hoặc khi thanh toán thành công) kéo theo nhu cầu hoàn kho khi khách bỏ thanh toán trực tuyến; cần cơ chế hết hạn đơn chưa thanh toán (ví dụ tác vụ định kỳ). Đây là điểm nên chốt với giảng viên.

Chương 4. Quy trình hiện thực hóa giao diện (Server-Side Rendering với EJS)

Giao diện **không dùng framework Frontend (React, Vue…)**. Thay vào đó, Express dựng trang HTML hoàn chỉnh ở máy chủ bằng **view engine EJS** (Embedded JavaScript templates) rồi gửi về trình duyệt. Mô hình này là **Server-Side Rendering (SSR)** theo kiểu MVC: Route nhận request → Controller lấy dữ liệu qua Model → Controller gọi `res.render` với một mẫu EJS và dữ liệu → trình duyệt nhận HTML.

Tại sao chọn SSR với EJS cho đồ án này?

- **SEO và tốc độ hiển thị lần đầu:** công cụ tìm kiếm và người dùng nhận ngay HTML có nội dung sản phẩm, rất quan trọng với cửa hàng bán hàng.
- **Đơn giản hóa hệ thống:** một ứng dụng, một đơn vị triển khai; không cần bộ công cụ build riêng, không phải xử lý CORS giữa hai miền, cookie xác thực dùng chung một nguồn gốc.
- **Giảm bề mặt tấn công:** logic nghiệp vụ và kiểm tra quyền nằm hoàn toàn ở máy chủ; trình duyệt chỉ nhận kết quả.
- **Đánh đổi cần nêu trong thuyết minh:** mỗi điều hướng tải lại cả trang; các tương tác phản hồi tức thời (chọn biến thể, thanh trượt giá) phải bổ sung bằng JavaScript thuần phía trình duyệt; máy chủ chịu thêm chi phí dựng trang.

4.1. Tổ chức tầng View

Thư mục	Nội dung	Mục đích
views/layouts	Khung trang chung (thẻ head, thanh điều hướng, chân trang)	Tránh lặp mã; một chỗ sửa cho mọi trang. EJS không có kế thừa mẫu sẵn nên dùng thư viện layout (ví dụ express-ejs-layouts) hoặc chèn partial.

Thư mục	Nội dung	Mục đích
views/partials	Mega Menu, thẻ sản phẩm, thanh phân trang, thông báo flash, ô chọn Size/Màu	Thành phần giao diện tái sử dụng; một thẻ sản phẩm dùng ở trang chủ, cửa hàng và gợi ý.
views/pages	Trang chủ, cửa hàng, chi tiết sản phẩm, giỏ hàng, thanh toán, đơn hàng, tài khoản, đăng nhập/đăng ký	Giao diện khách hàng.
views/admin	Trung tâm chỉ số, quản lý sản phẩm, danh mục, đơn hàng, người dùng	Giao diện quản trị, mọi route đi qua middleware RBAC.
public	CSS, JavaScript phía trình duyệt, ảnh giao diện	Tài nguyên tĩnh do Express phục vụ, kèm bộ nhớ đệm trình duyệt.

4.2. Cơ chế xử lý trang và biểu mẫu

- **Xem trang (GET):** Controller truy vấn dữ liệu và truyền một đối tượng dữ liệu cho mẫu EJS. Mẫu chỉ chứa logic hiển thị (vòng lặp, điều kiện), không truy vấn cơ sở dữ liệu.
- **Thao tác ghi (POST):** áp dụng mẫu **Post – Redirect – Get (PRG)**: xử lý POST xong thì chuyển hướng (redirect) về một trang GET. *Lý do:* tránh gửi lại biểu mẫu khi khách làm mới trang, chẳng hạn đặt hàng hai lần.
- **Thông báo (flash message):** kết quả thao tác (thêm vào giỏ, hết hàng, lỗi nhập) lưu tạm trong phiên và hiển thị một lần ở trang sau khi chuyển hướng.
- **Phương thức PUT/DELETE:** biểu mẫu HTML chỉ có GET và POST, nên dùng cơ chế ghi đè phương thức (method override) để giữ đúng ngữ nghĩa RESTful cho cập nhật và xóa.
- **Chống CSRF:** vì xác thực dựa trên cookie, **mọi biểu mẫu POST** phải mang mã chống CSRF do máy chủ phát; request thiếu hoặc sai mã bị từ chối. Cookie phiên đặt SameSite làm lớp bảo vệ bổ sung.
- **Tuyến JSON bổ trợ:** các tuyến dưới tiền tố /api vẫn trả JSON cho tương tác nhỏ (danh sách Quận/Huyện, kiểm tra tồn kho) và cho công cụ kiểm thử như Postman.

4.3. Giao diện phía khách hàng (Customer UI)

Trang	Controller truyền cho View	Cách làm và tương tác
Trang chủ	Cây danh mục cho Mega Menu; bộ sưu tập theo mùa; danh mục nổi bật và sản phẩm mới	Mega Menu dựng từ dữ liệu danh mục (có thể lưu đệm vì ít đổi); Slider/Carousel bằng CSS hoặc một thư viện nhẹ.

Trang	Controller truyền cho View	Cách làm và tương tác
Cửa hàng và bộ lọc	Danh sách sản phẩm đã lọc, tổng số bản ghi, trang hiện tại, giá trị bộ lọc đang chọn	Sidebar là một **biểu mẫu GET**: Checkbox Size, ô chọn Màu, khoảng giá. Gửi biểu mẫu thì tham số năm trên URL, máy chủ lọc và dựng lại lưới. Trạng thái bộ lọc được **điền lại** trong mẫu (ô đã chọn giữ nguyên). Phân trang là các liên kết giữ nguyên tham số lọc.
Chi tiết sản phẩm	Sản phẩm kèm toàn bộ biến thể và tồn kho	Máy chủ nhúng danh sách biến thể vào trang dưới dạng JSON an toàn; một đoạn JavaScript nhỏ, khi khách chọn Màu, **làm mờ** các Size có tồn kho 0. Máy chủ vẫn kiểm tra lại khi thêm vào giỏ. Zoom ảnh bằng thư viện nhẹ hoặc CSS.
Giỏ hàng	Các dòng hàng (tên, ảnh, Size/Màu, số lượng, đơn giá), tổng tiền	Guest dùng giỏ lưu trong **phiên (session)** phía máy chủ; khi đăng nhập thì **gộp** vào giỏ lưu trên cơ sở dữ liệu. Thay đổi số lượng và xóa là biểu mẫu POST, máy chủ tính lại tổng và kiểm tra tồn kho.
Thanh toán	Giỏ hàng đã kiểm tra, địa chỉ mặc định, phương thức thanh toán	Chọn Tỉnh/Thành, Quận/Huyện, Phường/Xã phụ thuộc bậc: JavaScript nhỏ gọi tuyến /api để nạp danh sách cấp dưới. Thanh toán trực tuyến chuyển hướng sang cổng thanh toán; kết quả xác nhận bằng thông báo máy chủ – máy chủ (webhook), không tin trang trả về trên trình duyệt.
Đơn hàng của tôi	Danh sách đơn và chi tiết của chính khách (kiểm tra quyền sở hữu)	Nút Hủy đơn chỉ hiện khi trạng thái cho phép; máy chủ vẫn kiểm tra lại.

Lập luận bổ sung. Với SSR, **phân trang truyền thống bằng liên kết** là lựa chọn tự nhiên: mỗi trang có URL riêng, hỗ trợ SEO và nút Quay lại. Cuộn vô hạn chỉ nên thêm như một cải tiến tùy chọn bằng JavaScript nạp từng đợt.

4.4. Giao diện điều hành (Admin UI)

- **Trung tâm chỉ số:** máy chủ tổng hợp (aggregation) doanh thu theo thời gian, tỷ trọng danh mục bán chạy, số đơn cần xử lý gấp và các biến thể chạm ngưỡng tồn kho, rồi truyền cho mẫu. Biểu đồ đường và biểu đồ tròn vẽ bằng một thư viện biểu đồ chạy ở trình duyệt (ví dụ Chart.js) nhận dữ liệu JSON do máy chủ nhúng vào trang.
- **Quản lý sản phẩm:** bảng dữ liệu hỗ trợ tìm kiếm và lọc qua tham số URL. Biểu mẫu Thêm/Sửa có **dòng biến thể động**: nút "Thêm biến thể" dùng JavaScript thuần để nhân bản một hàng gồm Size – Màu – Số lượng kho, các trường đặt tên theo dạng mảng để máy chủ gom thành mảng biến thể. Máy chủ kiểm tra **không trùng cặp Size – Màu** và số lượng không âm. Ảnh tải lên qua biểu mẫu nhiều phần (multipart), máy chủ đẩy lên dịch vụ lưu trữ đám mây và chỉ lưu URL.
- **Điều phối đơn hàng:** các tab trạng thái là liên kết có tham số trạng thái; nút chuyển trạng thái chỉ hiện các bước tiếp theo hợp lệ theo máy trạng thái ở mục 1.3; mỗi lần chuyển, máy chủ gửi email thông báo cho khách.

Lập luận bổ sung. Ẩn nút ở giao diện chỉ là tiện lợi, **không phải bảo mật**: mọi route quản trị vẫn phải được chặn bằng RBAC ở máy chủ, người không có quyền nhận 403.

4.5. Bảo mật và hiệu năng của tầng View

- **Thoát ký tự đầu ra:** dùng cú pháp xuất có thoát ký tự tự động của EJS cho mọi dữ liệu người dùng (tên, địa chỉ, đánh giá). Cú pháp xuất thô chỉ dùng cho nội dung đã tin cậy. Đây là tuyến phòng thủ chính chống XSS, bổ sung cho Sanitize Middleware.
- **Chính sách nội dung (CSP):** cấu hình qua Helmet; vì có JavaScript nhúng trong trang nên cần dùng mã nonce thay vì cho phép script nội tuyến tùy ý.
- **Hiệu năng:** bật bộ nhớ đệm mẫu của EJS ở môi trường Production, nén phản hồi, bộ nhớ đệm HTTP cho tài nguyên tĩnh, lưu đệm các khối ít đổi (Mega Menu, trang chủ).

> **[Cần thẩm định]** (1) Thư viện chống CSRF phổ biến trước đây (csurf) theo hiểu biết của tôi đã bị ngừng duy trì; tôi chưa tra cứu nguồn hiện hành nên cần kiểm tra và chọn giải pháp còn được hỗ trợ. (2) Dữ liệu hành chính cho danh sách Tỉnh/Quận/Phường có nguy cơ lỗi thời (xem mục A1 phụ lục). (3) Lưu giỏ hàng Guest trong phiên máy chủ cần kho phiên bền (ví dụ lưu trên MongoDB) khi chạy nhiều tiến trình; kho phiên mặc định trong bộ nhớ không phù hợp Production.

Chương 5. Kiểm thử, tối ưu và triển khai hệ thống

5.1. Kế hoạch kiểm thử

Kế hoạch kiểm thử đi từ chức năng đến bảo mật, mỗi kịch bản có **kết quả mong đợi đo được** để dùng làm minh chứng trong thuyết minh.

Nhóm kiểm thử	Kịch bản	Kết quả mong đợi
End-to-End: luồng nghiệp vụ đóng	Tạo tài khoản khách mới → lọc và chọn mua một sản phẩm với Size cụ thể → đặt hàng thành công → đăng nhập Admin xác nhận đơn → kiểm tra tồn kho và trạng thái ở lịch sử đơn của khách	Tồn kho của đúng biến thể giảm đúng số lượng mua; trạng thái đơn đổi đồng bộ ở hai phía; giá trong đơn bằng giá lúc mua.
Phân quyền (RBAC)	Dùng Postman gửi lệnh xóa sản phẩm bằng tài khoản Customer, và bằng request không có cookie	Customer nhận **403 Forbidden**; request không đăng nhập nhận **401 Unauthorized**; dữ liệu không đổi.
Sở hữu dữ liệu (IDOR)	Customer A gọi xem hoặc hủy đơn của Customer B bằng cách đổi mã đơn	Bị từ chối (403 hoặc 404); không lộ nội dung đơn.
XSS	Nhập chuỗi chứa thẻ script vào form đánh giá và form địa chỉ	Mã độc bị loại bỏ ở Backend và không thực thi khi trang EJS hiển thị lại dữ liệu đó.
Đồng thời tồn kho	Với biến thể còn 1 sản phẩm, gửi nhiều request đặt hàng gần như cùng lúc bằng công cụ tải	Đúng 1 đơn thành công, các đơn còn lại nhận lỗi hết hàng; tồn kho cuối bằng 0, không âm.
Bộ lọc	Lọc theo Size M và Màu Đỏ khi chỉ có biến thể M–Xanh và S–Đỏ	Sản phẩm đó không xuất hiện (kiểm chứng điều kiện kép trên cùng phần tử mảng).
CSRF	Gửi biểu mẫu POST (ví dụ đặt hàng, hủy đơn) từ một trang bên ngoài khi đang đăng nhập, không kèm mã chống CSRF	Request bị từ chối (403); đơn không được tạo và trạng thái không đổi.

Lập luận bổ sung. Kịch bản "đồng thời tồn kho" là minh chứng thuyết phục nhất cho giải pháp ở mục 3.4, nên nên có ảnh chụp kết quả trước và sau khi áp dụng phép trừ nguyên tử để so sánh. Nên bổ sung kiểm thử đơn vị cho hàm tính giá và hàm dựng điều kiện lọc, và

kiểm thử API tự động chạy trong quy trình tích hợp liên tục (CI).

5.2. Tối ưu hiệu năng

- **Cơ sở dữ liệu:** các chỉ mục ở mục 2.3; chỉ lấy các trường cần dùng cho danh sách (không trả toàn bộ mô tả và mảng ảnh).
- **Mạng:** nén phản hồi, ảnh qua CDN với kích thước phù hợp, bộ nhớ đệm cho dữ liệu ít đổi như cây danh mục.
- **Giao diện SSR:** bộ nhớ đệm HTTP cho tài nguyên tĩnh, tải chậm ảnh (lazy loading), lưu đệm các khối trang ít đổi như mục 4.5.

5.3. Chiến lược triển khai

Thành phần	Nền tảng	Công việc chính	Lý do và lưu ý
Cơ sở dữ liệu	MongoDB Atlas	Chuyển dữ liệu từ máy cục bộ lên cụm đám mây; tạo tài khoản truy cập quyền tối thiểu; cấu hình **IP Access List**	Có sẵn sao lưu và bộ bản sao (cần cho giao dịch). Khi Backend dùng IP động (Render/Railway gói thấp), danh sách IP thường phải mở rộng; cần cân nhắc rủi ro.
Ứng dụng web (Express + EJS)	Render hoặc Railway	Dựng từ mã nguồn; đặt biến môi trường Production; bật tự khởi động lại (Auto-restart); cấu hình điểm kiểm tra sức khỏe (health check)	Gói miễn phí có thể ngủ khi không có truy cập, gây request đầu chậm.
Tài nguyên tĩnh và ảnh	Do chính ứng dụng web phục vụ; ảnh sản phẩm trên dịch vụ lưu trữ đám mây	Express phục vụ thư mục tài nguyên tĩnh (CSS, JavaScript, ảnh giao diện) kèm bộ nhớ đệm; ảnh sản phẩm tải lên dịch vụ lưu trữ đám mây và chỉ lưu URL	Chỉ còn một đơn vị triển khai nên không còn bài toán CORS giữa hai miền; cần bật cấu hình tin cậy proxy (trust proxy) để cookie Secure hoạt động đúng sau hạ tầng của nền tảng.

> **[Cần thẩm định]** Tên gói, giới hạn gói miễn phí và chính sách của Render, Railway, Atlas thay đổi thường xuyên; tôi chưa tra cứu nguồn hiện hành cho các chi tiết này, nên các nhận xét về gói miễn phí ở trên là hiểu biết chung, cần kiểm chứng trước khi đưa vào thuyết minh.

Phụ lục. Ghi chú thẩm định (dành cho bước rà soát)

Phần này tách riêng **kết quả đã chốt** khỏi **điểm chưa chắc chắn** để người rà soát kiểm tra nhanh. Có thể xóa phụ lục trước khi nộp.

A. Điểm chưa chắc chắn hoặc chưa kiểm chứng

Mã	Nội dung	Mức độ	Hành động đề xuất
A1	Dữ liệu đơn vị hành chính ba cấp có thể đã lỗi thời sau đợt sắp xếp năm 2025; chưa tra nguồn	Cao	Xác nhận mô hình địa chỉ với giảng viên và kiểm tra nguồn dữ liệu
A2	Gói miễn phí, giới hạn và hành vi của Render, Railway, Atlas chưa tra cứu hiện hành	Trung bình	Tra tài liệu chính thức trước khi trích dẫn
A3	Cấu hình cookie Secure khi chạy sau proxy của nền tảng triển khai, và việc chọn giải pháp chống CSRF còn được hỗ trợ	Trung bình	Thử nghiệm trên môi trường thật sau khi triển khai
A4	Việc dùng DOMPurify trên Node.js cần môi trường DOM; hiệu quả của làm sạch đầu vào so với mã hóa đầu ra	Trung bình	Cân nhắc thư viện chuyên cho Node và bắt buộc thoát ký tự đầu ra ở tầng View EJS
A5	Thời điểm trừ kho (khi đặt, khi xác nhận hay khi thanh toán)	Trung bình	Chốt quy tắc nghiệp vụ, thêm cơ chế hết hạn đơn chưa thanh toán

B. Nội dung tôi bổ sung ngoài khung sườn gốc

- Trạng thái "Đã xác nhận" và máy trạng thái đơn hàng (mục 1.3).
- Bước phân quyền RBAC, kiểm tra đầu vào, xử lý lỗi tập trung và giới hạn tần suất trong Request Lifecycle (mục 2.2).
- Thực thể Cart và Review, danh sách chỉ mục, mã SKU cho biến thể (mục 2.3).

- Token làm mới, thoát ký tự biểu thức chính quy, giao dịch đa tài liệu, kiểm tra quyền sở hữu (mục 3.2 – 3.4, 1.1).
- Kịch bản kiểm thử IDOR, CSRF, đồng thời tồn kho và bộ lọc (mục 5.1). Giao diện chuyển sang SSR với EJS theo yêu cầu mới (Chương 4).

C. Điểm có thể tối ưu thêm

- Thay phân trang bỏ qua/giới hạn bằng phân trang theo con trỏ khi dùng cuộn vô hạn.
- Dùng chỉ mục văn bản hoặc Atlas Search thay biểu thức chính quy cho tìm kiếm từ khóa.
- Đưa thanh toán trực tuyến qua cơ chế xác nhận bằng webhook của cổng thanh toán, thay vì tin kết quả do trình duyệt báo về.
- Thêm ghi nhật ký kiểm toán (audit log) cho thao tác của Admin như đổi trạng thái đơn, đổi vai trò.
- Tài liệu hóa API bằng đặc tả OpenAPI để phần JavaScript ở trình duyệt và kiểm thử dùng chung.

D. Hướng dẫn chỉnh cho đúng văn phong bài nộp

Báo cáo này không chứa mã nguồn theo yêu cầu. Khi đưa vào thuyết minh, nên thêm các hình vẽ chính thức (sơ đồ use case, sơ đồ trình tự cho đặt hàng, sơ đồ thực thể – quan hệ cho dữ liệu) dựng từ các mô tả ở Chương 1 và 2, và đánh số hình, bảng theo quy định của khoa.